

IAM Role

An IAM Role is an identity you can create that has specific permissions with credentials that are valid for a short duration. The role can be assumed by an entity that you trust.

Key Idea: Unlike an IAM user:

-  No username/password
-  No permanent credentials
-  Provides **temporary credentials**

Where Roles Are Used

1. EC2 Instances

- Attach a role to an EC2 instance
- An instance can access services (like S3) without storing keys

Example: EC2 reads files from S3 using a role.

2. **Cross-Account Access** - One AWS account can access another using roles
3. **Services Access** - AWS services (like Lambda, ECS) assume roles to perform actions

How It Works

1. You create a role
2. Attach a policy (permissions)
3. Define who can assume it
4. That entity uses the role temporarily

Roles V/S Users

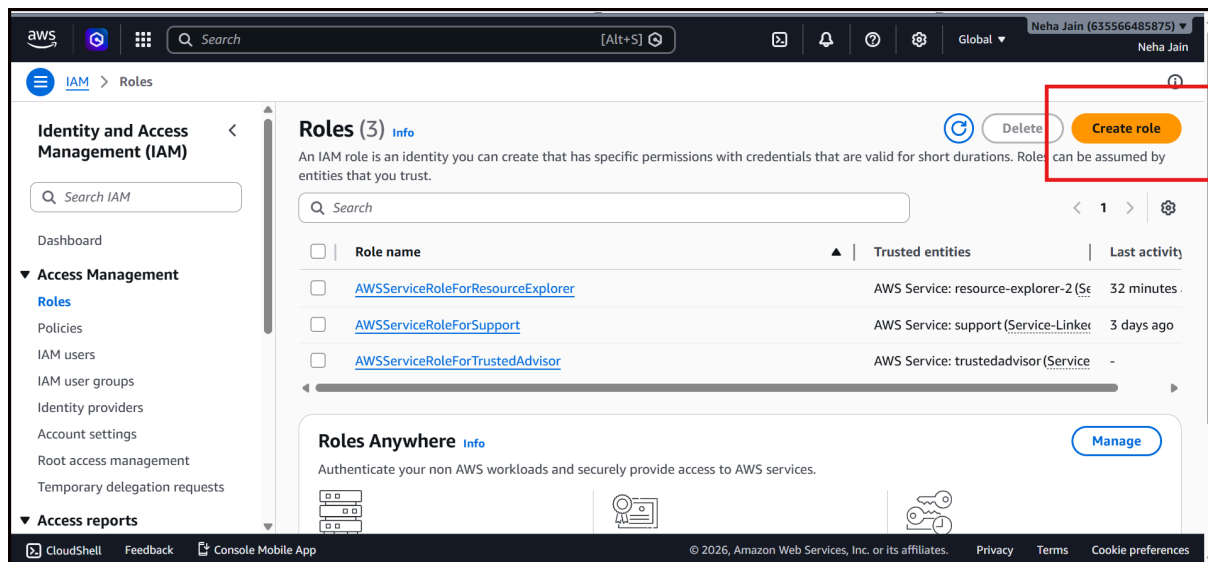
◆ IAM User vs Role

Feature	IAM User	IAM Role
Credentials	Permanent	Temporary
Login	Yes	No
Use case	Humans/apps	Services, temporary access
Security	Less secure	More secure

Show me how to create a role step-by-step

Step 1: Open IAM

- ☐ Go to AWS Console
- ☐ Search → IAM
- ☐ Click Roles (left panel)



Step 2: Create Role

Click **Create role**

Step 1: Select trusted entity

Step 2: Add permissions

Step 3: Name, review, and create

Select trusted entity info

Trusted entity type

- ☒ **AWS service**
Allow AWS services like EC2, Lambda, or others to perform actions in this account.
- ☐ **AWS account**
Allow entities in other AWS accounts belonging to you or a 3rd party to perform actions in this account.
- ☐ **Web identity**
Allows users federated by the specified external web identity provider to assume this role to perform actions in this account.
- ☐ **SAML 2.0 federation**
Allow users federated with SAML 2.0 from a corporate directory to perform actions in this account.
- ☐ **Custom trust policy**
Create a custom trust policy to enable others to perform actions in this account.

Step 3: Select Trusted Entity

- Choose: ☒ **AWS service**
- Under use case:
Select **EC2**
- Click **Next**

Step 1: Select trusted entity

Step 2: Use case

Step 3: Name, review, and create

Use case

Allow an AWS service like EC2, Lambda, or others to perform actions in this account.

Service or use case

EC2

Choose a use case for the specified service.

Use case

- ☒ **EC2**
Allows EC2 instances to call AWS services on your behalf.
- ☐ **EC2 Role for AWS Systems Manager**
Allows EC2 instances to call AWS services like CloudWatch and Systems Manager on your behalf.
- ☐ **EC2 Spot Fleet Role**
Allows EC2 Spot Fleet to request and terminate Spot Instances on your behalf.
- ☐ **EC2 - Spot Fleet Auto Scaling**
Allows Auto Scaling to access and update EC2 spot fleets on your behalf.
- ☐ **EC2 - Spot Fleet Tagging**
Allows EC2 to launch spot instances and attach tags to the launched instances on your behalf.
- ☐ **EC2 - Spot Instances**

Step 4: Attach Permissions:

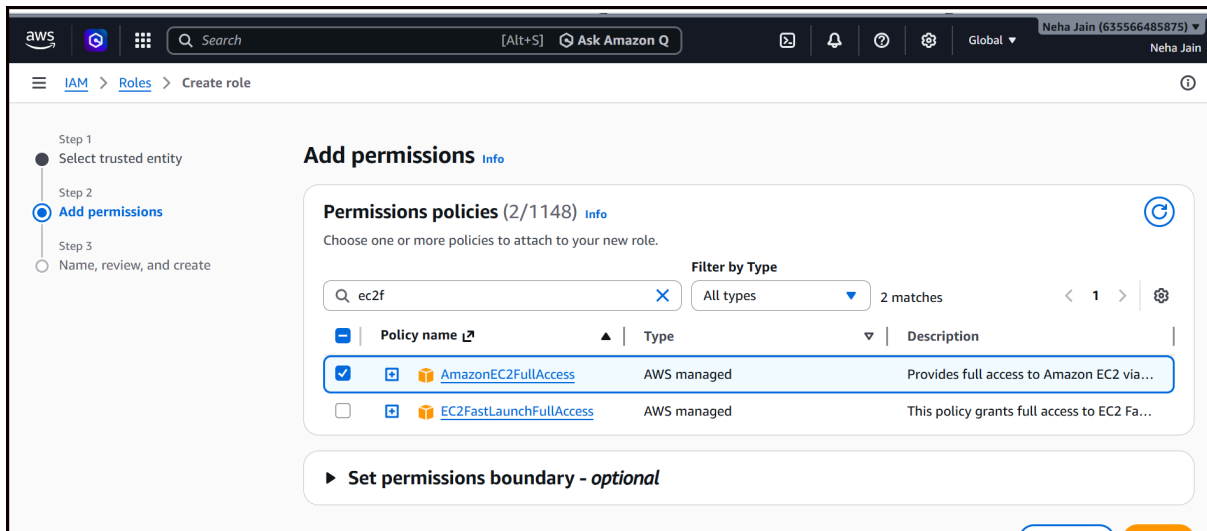
Choose a policy based on your needs

Example:

- **AmazonS3ReadOnlyAccess** → if EC2 needs S3 access
- **AmazonEC2FullAccess** → (not recommended for prod)

👉 Or attach your custom policy

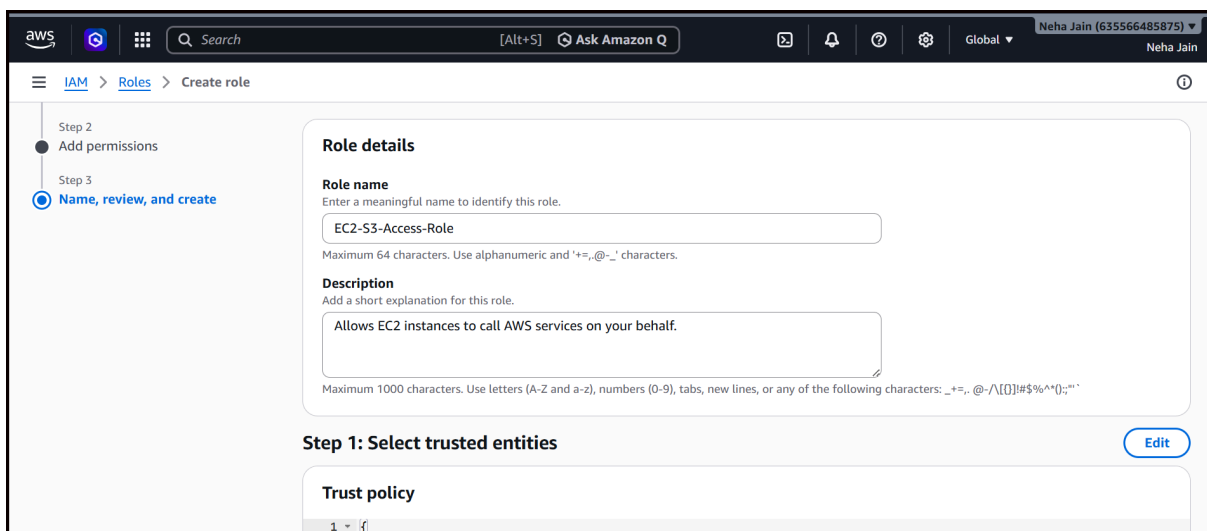
Click Next



Step 5: Name the Role

Enter:

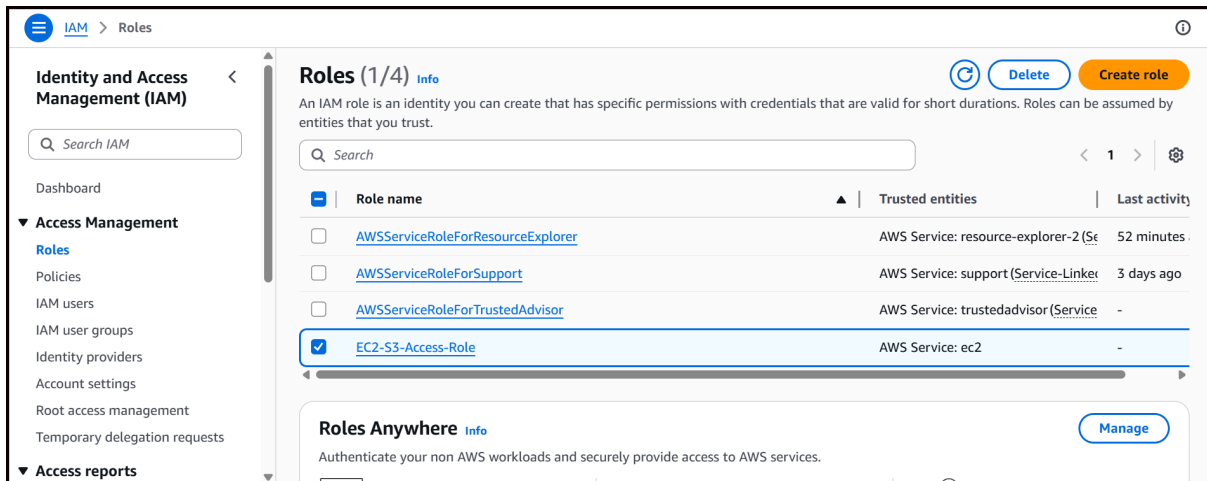
Role name → **EC2-S3-Access-Role**



Step 6: Attach Role to EC2 Instance

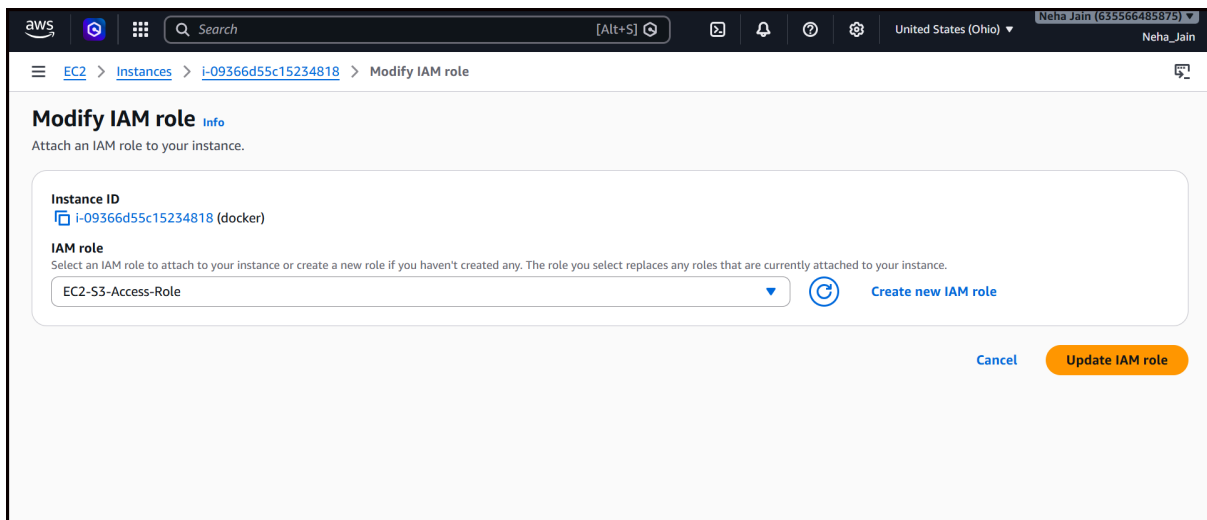
Option A: While launching an instance

- In EC2 launch wizard:
 - Under IAM role
 - Select your role



Option B: Attach to existing instance

- Go to EC2 → Instances
- Select instance
- Click:
 - Actions → Security → Modify IAM role
- Select your role
- Save



Step-by-Step: Access S3 from EC2 using Role

Step 1: Connect to your EC2 instance

```
ssh -i "docker.pem"
ubuntu@ec2-3-136-84-255.us-east-2.compute.amazonaws.com
```

Step 2: Verify the role is active -

```
aws sts get-caller-identity
```

```
{
  "UserId": "AROAZH6VW6FZ33J60AJFH:i-09366d55c15234818",
  "Account": "635566485875",
  "Arn":
    "arn:aws:sts::635566485875:assumed-role/EC2-S3-Access-Role/i-09366d55c15234818"
}
```

✓ This confirms your EC2 is using the IAM role

Step 3: Access S3 (no credentials needed)

```
aws s3 ls
2026-04-28 21:44:55 demo-quiz-app
```

List files inside a bucket:

```
aws s3 ls s3://demo-quiz-app
2026-01-30 18:08:00      3000 index.html
2026-01-28 18:01:19      1802 questions.js
2026-01-28 18:15:30     10183 script.js
2026-01-28 18:01:21      7432 style.css
```

Download a file:

```
aws s3 cp s3://your-bucket-name/file.txt .
```

Upload a file:

```
aws s3 cp test.txt s3://your-bucket-name/
```

```
ubuntu@ip-172-31-21-40:~$ aws sts get-caller-identity
{
  "UserId": "AROAZH6VW6FZ33J60AJFH:i-09366d55c15234818",
  "Account": "635566485875",
  "Arn": "arn:aws:sts::635566485875:assumed-role/EC2-S3-Access-Role/i-09366d55c15234818"
}
ubuntu@ip-172-31-21-40:~$ aws s3 ls
2026-04-28 21:44:55 demo-quiz-app
ubuntu@ip-172-31-21-40:~$ aws s3 ls s3://demo-quiz-app
2026-01-30 18:08:00      3000 index.html
2026-01-28 18:01:19      1802 questions.js
2026-01-28 18:15:30     10183 script.js
2026-01-28 18:01:21      7432 style.css
```